



ELEKTROTEHNIŠKO-RAČUNALNIŠKA
STROKOVNA ŠOLA IN GIMNAZIJA
LJUBLJANA

Vegova ulica 4, 1000 Ljubljana

Splošna matura
Seminarska naloga pri računalništvu

NADGRADNJA DALJINSKO VODENE KOSILNICE Z RAČUNALNIŠKIM VIDOM IN PROSTORSKIM UMEŠČANJEM

Mentor: Matjaž Majnik

Avtor: Lin Čadež, G 4. A

Ljubljana, april 2026

Zahvala

Zahvaljujem se mentorju Matjažu Majniku za pomoč in vodenje pri izdelavi seminarske naloge. Posebna zahvala gre mojemu očetu, ki mi je pomagal s finančno podporo pri nakupu GNSS modula in PiCama ter za potrpežljivost in zaupanje med testiranjem mojih novih idej na njegovem mulčerju.

Povzetek

V seminarski nalogi sem nadgradil daljinsko vodeno kosilnico tako, da sem jo poskušal narediti varnejšo in bolj pregledno za operaterja. Izhajal sem iz konkretnega problema: mulčer je težak, ima ostra rezila in ga upravljam na daljavo, zato sem želel dodati sistem, ki zazna človeka ali žival in stroj samodejno ustavi. Sprva sem preizkušal enostavnejše senzorje, vendar sem ugotovil, da ne ločijo dovolj zanesljivo med ovirami in živimi bitji, zato sem se osredotočil na kamero, računalniški vid in MobileNetSSD. Hkrati sem preveril PWM-sigle daljinca, dodal spletni vmesnik za video in položaj ter preizkusil DGPS z NTRIP-popravki. V nalogi sem pokazal, kaj sem v kodi dejansko zgradil, zakaj sem izbral določene rešitve in kje so ostale omejitve.

Ključne besede: daljinsko vodena kosilnica, računalniški vid, PWM, DGPS, NTRIP, Raspberry Pi

Kazalo

1	Uvod	6
2	Jedro	7
2.1	Daljinsko vodene kosilnice in varnostni izzivi	7
2.2	Elektronsko zaznavanje okolice pri delovnih strojih	7
2.3	Pregled tehnologij	8
2.3.1	Raspberry Pi 4B	9
2.3.2	OpenCV in MobileNetSSD	10
2.3.3	Picamera2	11
2.3.4	GNSS modul LC29H z DGPS podporo	11
2.3.5	NTRIP in RTK2go	11
2.3.6	Python in večnitnost	11
2.4	Analiza RC-krmiljenja in PWM-meritve	11
2.4.1	Varnostna preklopna logika	12
2.5	Načrt sistema in razvojne iteracije	12
2.5.1	Osnovna arhitektura sistema	12
2.5.2	Nastavitve za RC, kamero in GNSS	13
2.5.3	Glavna zanka programa	13
2.5.4	Strežnik in uporabniški vmesnik	14
2.6	Varnostni mehanizem samodejne zaustavitve	15
2.6.1	Sprotna zaustavitev ob zaznavi človeka	15
2.6.2	Clear-delay logika	16
2.6.3	Zaznava objektov s MobileNetSSD	16
2.6.4	Zmanjšanje zakasnitve video toka	16
2.7	GNSS, DGPS in NTRIP v praksi	17
2.7.1	Delovanje GPS in omejitve navadnih modulov	17
2.7.2	Diferencialni GPS in NTRIP	17
2.7.3	Povezava na NTRIP in pošiljanje GGA	18
2.8	Spletni vmesnik za operaterja	19
2.9	Testiranje sistema in analiza rezultatov	19
3	Zaključek	22
4	Viri in literatura	23

Slike

1	Izhodiščni mulčer, ki sem ga uporabil za nadgradnjo.	6
2	Vsi moduli sistema, povezani skupaj: GPS ščit, RC sprejemnik, PiCam, Raspberry Pi in GPS antena.	9
3	Shema priključkov sprejemnika iz dokumentacije, ki sem jo uporabil pri načrtovanju GPIO-vezave.	9
4	Daljinec z označenimi gumbi in joysticki, ki sem jih testiral in jih uporabljam pri upravljanju kosilnice. Stikalo na vrhu služi za spremembo načina delovanja, joysticka pa za usmerjanje naprej, nazaj, levo in desno.	10
5	Skica Raspberry Pi z označenimi pini, ki sem jih uporabljal pri povezovanju sistema.	10
6	Graf meritev PWM-signalov posameznih kanalov, izmerjenih z Arduino in Serial Plotterjem v Arduino IDE.	12
7	Prikaz principa delovanja diferencialnega GPS in popravkov NTRIP.	18
8	Spletni vmesnik z živim videoposnetkom kamere in GPS-lokacijo kosilnice na interaktivnem zemljevidu.	19
9	Testiranje sistema na terenu: mulčer od daleč s prenosnim računalnikom med preizkusi.	20
10	Notranjost mulčerja z dodanim Raspberry Pi računalnikom in nameščeno elektroniko.	21
11	Daljinec, nameščen na mulčerju med terenskim testiranjem.	22

1 Uvod

Daljinsko vodene kosilnice se vse pogosteje uporabljajo za košnjo težje dostopnih terenov, kjer je klasična ročna košnja zahtevna ali nevarna. Opremljene so z ostrimi rezili, ki jih poganja bencinski motor, gibanje pa omogočajo električni motorji in gosenični pogon. Zaradi tega lahko predstavljajo resno varnostno tveganje, zlasti če se v njihovi bližini znajdejo ljudje ali živali. Kljub daljinskemu upravljanju ostaja odprto vprašanje, kako zagotoviti dodatno varnost in zmanjšati možnost nesreč.

Za seminarsko nalogo sem se odločil na podlagi osebne izkušnje z uporabo daljinsko vodene kosilnice. Med uporabo sem razmišljal, kako bi bilo mogoče izboljšati varnost njenega delovanja. Sprva sem preizkušal preproste ultrazvočne senzorje za zaznavanje ovir, vendar sem ugotovil, da ne omogočajo zanesljivega razlikovanja med predmeti ter ljudmi in živalmi. Zato sem se odločil za uporabo kamere in računalniškega vida, ki omogočata natančnejše zaznavanje okolice.

Namen seminarske naloge je predelava navadne daljinsko vodene kosilnice v varnejši sistem z zaznavanjem okolice in prostorsko umestitvijo. Cilj je razviti sistem, ki ob zaznavi človeka ali živali samodejno zaustavi kosilnico ter uporabniku omogoča nadzor nad njenim delovanjem in položajem, pri čemer osnovno daljinsko upravljanje ostane nespremenjeno.



Slika 1: Izhodiščni mulčer, ki sem ga uporabil za nadgradnjo.

Cilji seminarske naloge so:

- analizirati delovanje daljinskega upravljanja kosilnice in PWM-signalov,
- izbrati primeren sistem za zaznavanje ljudi in živali z uporabo kamere,
- razviti varnostni mehanizem za samodejno zaustavitev kosilnice,

- preizkusiti prostorsko umestitev kosilnice z uporabo diferencialnega GPS,
- izdelati osnovni spletni vmesnik za spremljanje delovanja sistema.

Hipoteza seminarske naloge je, da je z uporabo računalniškega vida mogoče zanesljivo zaznati ljudi in živali v okolici kosilnice ter s tem preprečiti morebitne poškodbe.

Raziskovalne metode so temeljile na eksperimentalnem delu, meritvah signalov, praktičnih preizkusih delovanja sistema in analizi rezultatov testiranj na terenu.

2 Jedro

2.1 Daljinsko vodene kosilnice in varnostni izzivi

Daljinsko vodene kosilnice so posebna vrsta delovnih strojev, namenjenih košnji trave in vzdrževanju površin, kjer bi bila klasična košnja z ročno ali traktorsko kosilnico neugodna oziroma celo nevarna, na primer na strmih pobočjih, v trnju ali na drugih težje dostopnih terenih. Upravljanje poteka z brezžičnim daljinskim upravljalnikom, kar omogoča, da se operater nahaja na varni razdalji od stroja. Takšne kosilnice se pogosto uporabljajo na strmih pobočjih, ob cestah, na brežinah in drugih zahtevnih terenih, kjer obstaja večja nevarnost zdrsa ali izgube nadzora nad strojem.

Delovanje daljinsko vodenih kosilnic temelji na kombinaciji mehanskega in električnega pogona. Rezalni sistem običajno poganja bencinski motor, ki omogoča visoko moč in učinkovito košnjo, medtem ko za premikanje in krmiljenje pogosto skrbijo elektromotorji, povezani z goseničnim ali kolesnim pogonom. Ukazi z daljinskega upravljalnika se do kosilnice prenašajo v obliki PWM-signalov, ki določajo smer, hitrost gibanja ter druge funkcije stroja.

Kljub temu pa takšne kosilnice predstavljajo resne varnostne izzive. Največjo nevarnost predstavljajo hitro vrteča se rezila, ki lahko ob stiku povzročijo hude telesne poškodbe. Dodatno tveganje izhaja iz samega gibanja stroja, saj lahko na neravnem ali strmem terenu pride do nenadnega zdrsa, prevračanja ali nenadzorovanega premika. Posebej nevarne so situacije, ko se v neposredni bližini kosilnice znajdejo ljudje ali živali, saj operater kljub daljinskemu upravljanju morda ne zazna pravočasno vseh ovir v okolici.

Zaradi teh nevarnosti se v praksi uporabljajo različni varnostni pristopi. Med najpogostejšimi so mehanski zaščitni pokrovi rezil, sistemi za takojšnje izklapljanje rezil ob določenih pogojih ter osnovni senzorji, ki zaznajo nagib ali dvig stroja. V strokovni literaturi je poudarjeno, da daljinsko upravljanje samo po sebi še ne zagotavlja popolne varnosti, temveč je potrebno varnostne mehanizme obravnavati kot sestavni del celotnega sistema [3].

V svojem projektu sem za to uporabil `rpicas-detect`, ki temelji na modelu MobileNet SSD in omogoča prepoznavanje ljudi, mačk, avtomobilov in drugih objektov [5].

2.2 Elektronsko zaznavanje okolice pri delovnih strojih

Pri delovanju avtonomnih ali daljinsko vodenih strojev je zanesljivo zaznavanje okolice ključno za varno navigacijo in izogibanje oviram. Različne vrste senzorjev omogočajo strojem, da zaznajo predmete, ovire in spremembe v okolju ter se nanje ustrezno odzovejo.

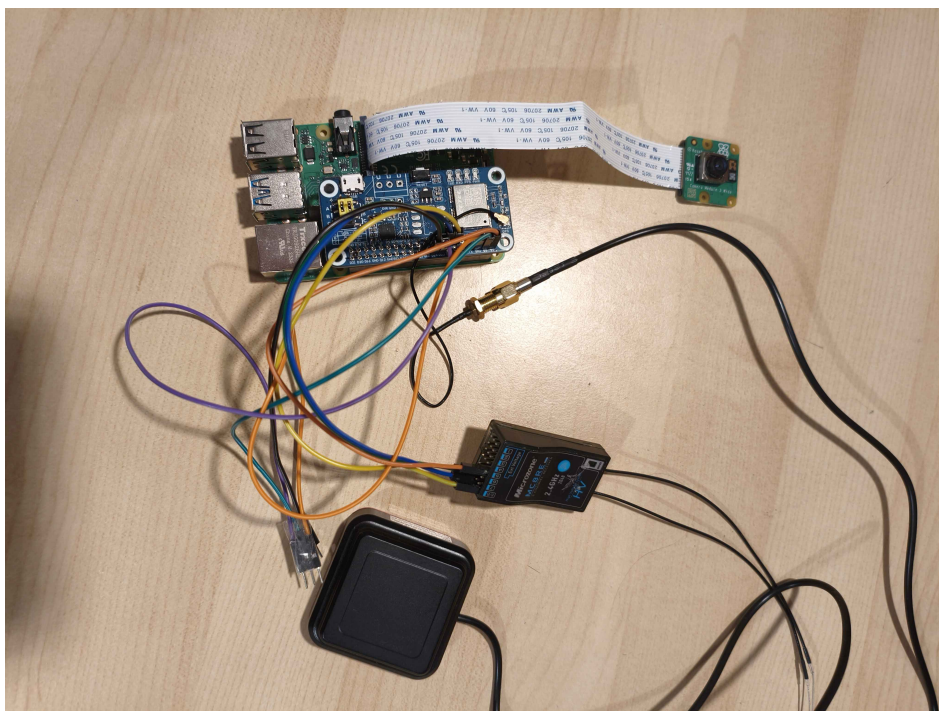
Ultrazvočni senzorji delujejo tako, da oddajajo zvočne valove nad človeškim slušnim območjem in merijo čas, ki ga potrebuje signal, da se odbije od ovire in se vrne do sprejemnika. Na podlagi tega izračunajo razdaljo do ovire, kar omogoča sprotno zaznavanje prisotnosti predmetov v bližini. Takšni senzorji so cenovno ugodni, vendar ne omogočajo zanesljivega razlikovanja med različnimi vrstami objektov.

Infrardeči senzorji zaznavajo spremembe v infrardečem svetlobnem spektru. Uporabni so za zaznavanje premikanja in prisotnosti predmetov v bližini, vendar imajo krajši doseg od ultrazvočnih sistemov in so bolj občutljivi na svetlobne razmere ter temperaturne spremembe. Za poletno košnjo takšni senzorji niso najbolj primerni, saj so lahko zunanje temperature zelo visoke, včasih celo primerljive ali višje od temperature človeškega telesa. Zaradi tega je razlikovanje med živim bitjem in segreto okolico manj zanesljivo.

Sistem zaznavanja s pomočjo kamer zajema vizualne podatke, ki jih procesira programska oprema za identificiranje, kategorizacijo ter določanje položaja objektov v okolici stroja. Kamere omogočajo bistveno bogatejše informacije o okolju kot enostavni senzorji, saj omogočajo detekcijo, sledenje in razlikovanje med različnimi vrstami ovir. Takšne zmogljivosti so ključne pri nalogah, kjer je treba razlikovati med premičnimi in nepremičnimi predmeti ter zaznavati ljudi ali živali.

2.3 Pregled tehnologij

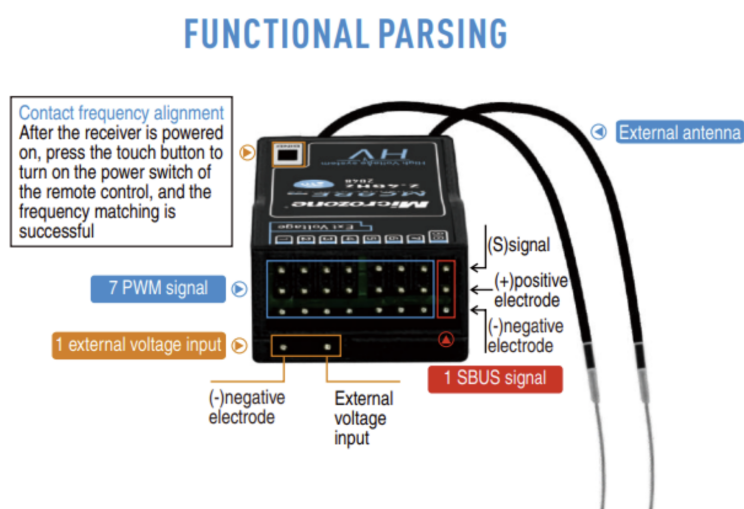
Pred začetkom razvoja sem moral izbrati ustrezne tehnologije za vsak podsistem. Iskal sem majhen in varčen računalnik, ki bi ga lahko enostavno vgradil v omejen prostor v škatli z elektroniko v kosilnici. Prav tako sem moral doseči, da je programska oprema dovolj hitra pri zaznavanju ovir. Zastavil sem si cilj, da od zaznave ovire do zaustavitve kosilnice ne sme preteči več kot 700 ms. Obenem sem si želel tudi skoraj trenutni prenos slike s kosilnice na spletni vmesnik. Tega cilja mi sicer ni uspelo doseči popolnoma, vendar sem zakasnitev zmanjšal na približno 1 do 1,1 s, kar je bilo za praktično uporabo še sprejemljivo.



Slika 2: Vsi moduli sistema, povezani skupaj: GPS ščit, RC sprejemnik, PiCam, Raspberry Pi in GPS antena.

2.3.1 Raspberry Pi 4B

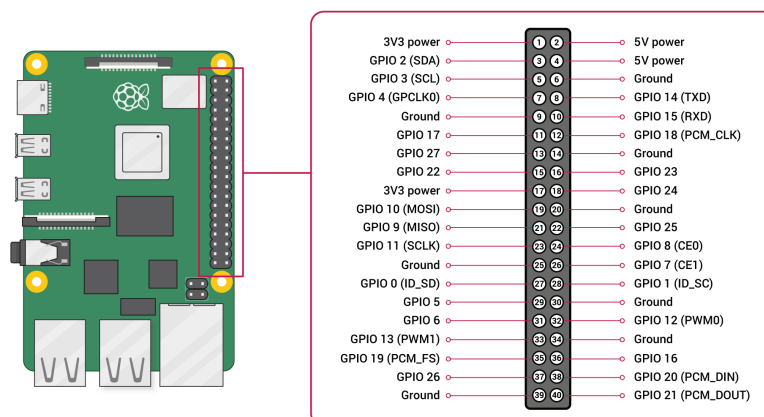
Raspberry Pi 4B je računalnik z vgrajenimi GPIO-priključki, CSI-vmesnikom za kamero, modulom Wi-Fi in operacijskim sistemom Linux (Raspberry Pi OS). Izbral sem ga, ker je dovolj majhen, da sem ga lahko namestil neposredno v kosilnico. Za njegovo delovanje zadostuje napetost 5 V, zato sem ga lahko priključil neposredno na elektroniko kosilnice, kjer sta bili na voljo napetosti 5 V in 12 V. Pomembna prednost je bila tudi dobra podpora za Python, OpenCV in delo s kamero, kar mi je bistveno olajšalo razvoj sistema.



Slika 3: Shema priključkov sprejemnika iz dokumentacije, ki sem jo uporabil pri načrtovanju GPIO-vezave.



Slika 4: Daljinec z označenimi gumbi in joysticki, ki sem jih testiral in jih uporabljam pri upravljanju kosilnice. Stikalo na vrhu služi za spremembo načina delovanja, joysticka pa za usmerjanje naprej, nazaj, levo in desno.



Slika 5: Skica Raspberry Pi z označenimi pini, ki sem jih uporabljal pri povezovanju sistema.

2.3.2 OpenCV in MobileNetSSD

OpenCV je odprtokodna knjižnica za računalniški vid, ki med drugim vsebuje modul `dnn` za poganjanje nevronske mreže. Za prepoznavanje objektov sem izbral med modelom YOLO, ki je natančnejši, vendar precej bolj računsko potraten, in modelom MobileNetSSD, ki je bil posebej zasnovan za naprave z omejeno procesorsko močjo. Na Raspberry Pi je bil MobileNetSSD

bistveno hitrejši in je dosegal zadovoljivo hitrost prepoznavanja. Nekaj težav z natančnostjo sem imel pri testih v temi, vendar se kosilnica v praksi uporablja predvsem podnevi, zato to ni bila ključna omejitev sistema.

2.3.3 Picamera2

Picamera2 je uradna Python knjižnica za upravljanje kamere na Raspberry Pi. V primerjavi s starejšo knjižnico `PiCamera` bolje podpira novejša kamerne module in se brezhibno integrira z OpenCV, kar omogoča neposreden prenos okvirjev v obliki NumPy matrik brez dodatnih konverzij [5].

2.3.4 GNSS modul LC29H z DGPS podporo

Za natančno pozicioniranje sem izbral modul LC29H podjetja Quectel, ki podpira sprejem signalov iz več satelitskih konstelacij (GPS, GLONASS, Galileo, BeiDou) in sprejem RTCM-diferencialnih popravkov prek UART-vmesnika. Navadni ceneni GPS-moduli dosegajo natančnost približno ± 5 m, moji testi z modulom NEO-7M, ki stane približno 6 €, pa so pokazali, da se lahko natančnost v neugodnih razmerah poslabša tudi na 30 m ali več. LC29H z DGPS-popravki dosega natančnost pod 1 m, kar je ključno za smiselno lociranje kosilnice pri delu na daljavo.

2.3.5 NTRIP in RTK2go

NTRIP (*Networked Transport of RTCM via Internet Protocol*) je standardiziran protokol za prenos diferencialnih GPS-popravkov prek interneta v realnem času. Za popravke sem uporabil javno dostopno storitev RTK2go in referenčno postajo FRELIH v Črnem Vrhu nad Idrijo [6]. Testiranja sem izvajal v Cerknem, ki je od postaje oddaljeno približno 33 km. Sicer bi bilo bolje, če bi bila referenčna postaja bližje, vendar sem bil med testiranjem zadovoljen z doseženo natančnostjo, ki je znašala približno $\pm 1,5$ m.

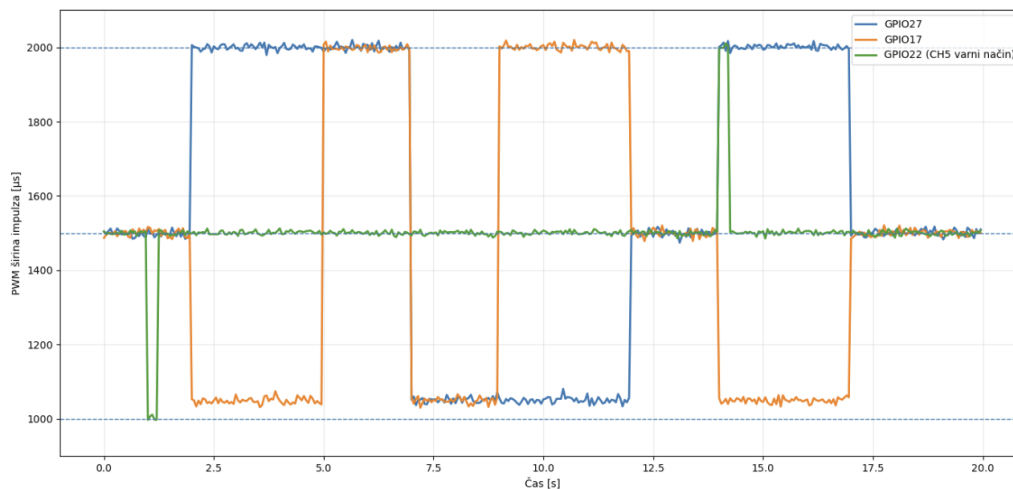
2.3.6 Python in večnitnost

Celoten sistem sem implementiral v Pythonu 3, ki je standardni jezik za razvoj na Raspberry Pi. Posamezne naloge, kot so branje kamere, video strežnik, branje GPS, pošiljanje NTRIP-podatkov in krmilna zanka, tečejo v ločenih nitih z modulom `threading`. Tak pristop omogoča, da sistem več opravil izvaja hkrati, kar je bistveno za dovolj hiter odziv pri delovanju v praksi.

2.4 Analiza RC-krmiljenja in PWM-meritve

Ko sem raziskoval daljinski upravljalnik, sem si najprej prebral dokumentacijo sprejemnika MC8RE [2]. Nato sem opravil meritve z Arduino Uno, ki mi je služil kot analogni merilec pulzov. Napisal sem program, ki je prek analognih pinov meril, s kakšno frekvenco sprejemnik oddaja PWM-signal, ki ga prejme od daljinca. Podatke sem prikazal v programu Arduino Serial Plotter znotraj okolja Arduino IDE. Tako sem ugotovil, da se posamezna smer prepozna po konkretnih pulznih širinah. Vizualizacija poteka pulzov v realnem času mi je omogočila, da sem lažje določil meje za histerezno logiko.

Pri meritvah sem dobil naslednji vzorec: premik naprej pomeni, da je eden od kanalov približno 2000 μs , drugi pa približno 1050 μs . Pri mirovanju sta oba kanala približno 1500 μs .



Slika 6: Graf meritev PWM-signalov posameznih kanalov, izmerjenih z Arduino in Serial Plotterjem v Arduino IDE.

2.4.1 Varnostna preklopna logika

Spodnji blok kode prikazuje, kako v glavni zanki preverjam, ali je vrednost kanala CH5, ki je vezan na stikalo na daljincu, večja od konstante `AUTOPILOT_ON`. V programu sem nastavil mejo za vklop na 1900 μs , mejo za izklop na 1100 μs , nevtralna vrednost pa je 1500 μs . Če se stikalo premakne le za kratek trenutek, se stanje nastavi na `True` ali `False`, nato pa se signal lahko vrne v nevtralno območje. Na ta način ostane način delovanja nastavljen glede na zadnji premik stikala.

```
if ch5 > AUTOPILOT_ON and not autopilot_mode:
    autopilot_mode = True
elif ch5 < AUTOPILOT_OFF and autopilot_mode:
    autopilot_mode = False
```

Izsek kode 1: Varnostna preklopna logika glede na kanal CH5.

2.5 Načrt sistema in razvojne iteracije

2.5.1 Osnovna arhitektura sistema

Najprej sem v glavni datoteki definiral osnovno arhitekturo sistema. Uvozil sem vse potrebne knjižnice, da sem lahko dosegel željeno funkcionalnost. Kot je razvidno iz spodnjega dela kode, sem uvozil tudi modul `threading`, saj je bilo zelo pomembno, da sistem hkrati izvaja več nalog: obdeluje sliko s kamere, oddaja video kot strežnik, po potrebi pošlje signal za zaustavitev ter sprejema in obdeluje GPS-koordinate skupaj s popravki.

```
import base64
import json
import socket
import threading
```

```

import time
from http.server import HTTPServer, BaseHTTPRequestHandler

import serial
import cv2
from picamera2 import Picamera2
import pigpio

```

Izsek kode 2: Osnovna struktura in knjižnice v final.py.

2.5.2 Nastavitve za RC, kamero in GNSS

Tu sem definiral osnovne konstante in preklopne meje na enem mestu, ker sem jih med testiranjem večkrat spreminjal.

V množici, to je podatkovni strukturi, podobni seznamu, vendar neurejeni, sem definiral imena objektov, ki jih nevronska mreža zazna na sliki. Z dodajanjem drugih imen objektov bi lahko dosegel, da bi se kosilnica ustavila tudi ob zaznavi drugih predmetov, na primer mobilnega telefona, vendar to za cilj moje naloge ni bilo potrebno.

```

CH1_IN = 17;  CH2_IN = 27;  CH5_IN = 22
CH1_OUT = 23; CH2_OUT = 24

MIN_PULSE = 900;  MAX_PULSE = 2100;  NEUTRAL_PULSE = 1500
AUTOPILOT_ON = 1900;  AUTOPILOT_OFF = 1100
CONF_THRESH = 0.5
TARGETS = {"person", "cat", "dog"}

```

Izsek kode 3: Ključne nastavitve za signalne pragove in zaznavo.

2.5.3 Glavna zanka programa

Glavna zanka programa povezuje vse podsisteme v enoten sistem. V funkciji `main()` se najprej odpre serijska povezava z GNSS-modulom, nato pa se za posamezne naloge zaženejo ločene niti. Ena nit skrbi za branje GNSS-podatkov, druga za povezavo z NTRIP-strežnikom, tretja za zajem slike s kamere, četrta za video tok, dodatna nit pa za spletni strežnik uporabniškega vmesnika. Na koncu se zažene `control_loop()`, ki neprekinjeno izvaja glavno krmilno logiko.

```

def main():
    ser = serial.Serial(SERIAL_PORT, SERIAL_BAUD, timeout=1)
    print("[INFO] GNSS serial open:", SERIAL_PORT, SERIAL_BAUD)

    threading.Thread(target=gnss_reader_thread, args=(ser,), daemon=
True).start()
    threading.Thread(target=ntrip_manager_thread, args=(ser,), daemon
=True).start()
    threading.Thread(target=camera_thread, daemon=True).start()
    threading.Thread(target=mjpeg_stream_thread, daemon=True).start()

    threading.Thread(

```

```

        target=lambda: HTTPServer(("0.0.0.0", HTTP_PORT),
DashboardHandler).serve_forever(),
        daemon=True
    ).start()

    control_loop()

```

Izsek kode 4: Glavna funkcija programa in zagon vseh niti.

Takšna zasnova omogoča, da sistem več funkcij izvaja hkrati in da posamezen podsistem ne blokira delovanja drugih.

2.5.4 Strežnik in uporabniški vmesnik

Pomemben del sistema je bil tudi spletni strežnik, ki operaterju omogoča dostop do slike s kamere in do trenutnega položaja kosilnice. Strežnik je pripravljen tako, da ob obisku osnovnega naslova vrne HTML-stran z uporabniškim vmesnikom, ob zahtevi na naslov `/pos` pa vrne trenutne GPS-podatke v obliki JSON. Na ta način lahko brskalnik sproti osvežuje položaj kosilnice in ga prikazuje na zemljevidu.

```

class DashboardHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/" or self.path == "/index.html":
            self.send_response(200)
            self.send_header("Content-Type", "text/html; charset=utf
-8")

            self.end_headers()
            self.wfile.write(INDEX_HTML)
            return

        if self.path == "/pos":
            with state_lock:
                data = json.dumps(latest).encode("utf-8")
                self.send_response(200)
                self.send_header("Content-Type", "application/json;
charset=utf-8")
                self.end_headers()
                self.wfile.write(data)
            return

```

Izsek kode 5: Osnovni del strežnika za pošiljanje HTML-strani in GPS-podatkov.

Na strani odjemalca sem uporabil JavaScript, ki v rednih časovnih presledkih pošilja zahteve na `/pos` in posodablja podatke na zemljevidu. Hkrati se naloži tudi video tok z naslova `/stream`, zato lahko operater na enem mestu spremlja sliko in lokacijo.

```

async function pollPos(){
    try{
        const r = await fetch('/pos', { cache:'no-store' });
        if(!r.ok) throw new Error('HTTP ' + r.status);
        const d = await r.json();
    }
}

```

```

    if(d.lat == null || d.lon == null){
        posPill.textContent = '/pos: no data';
        return;
    }

    marker.setLatLng([d.lat, d.lon]);
} catch(e) {
    posPill.textContent = '/pos: ERROR';
}
}

setInterval(pollPos, 500);

```

Izsek kode 6: Periodično pridobivanje položaja in posodabljanje zemljevida (JavaScript).

Takšna zasnova strežnika je bila zame primerna predvsem zato, ker je preprosta, pregledna in dovolj hitra za delovanje na Raspberry Pi.

Pomembno mi je bilo tudi, da so funkcije napisane tako, da sem lahko vsak podsistem testiral posebej. Tako sem najprej ločeno preveril delovanje kamere, GPS-a, NTRIP-povezave, spletnega vmesnika in RC-krmiljenja, šele nato pa sem vse dele združil v končno datoteko `final.py`.

2.6 Varnostni mehanizem samodejne zaustavitve

Pri varnostnem podsistemu sem najprej naredil osnovno različico, ki je kosilnico ustavila takoj, ko je kamera zaznala človeka, psa ali mačko. Opazil sem, da je imel sistem včasih težavo, da je oviro zaznal le v enem okvirju, zato se kosilnica ni vedno ustavila dovolj zanesljivo. Zato sem dodal časovni zamik, kar pomeni, da kosilnica po zaznavi ovire obstane še eno sekundo in šele nato ponovno preveri, ali je objekt še vedno pred njo. Če objekt ni več prisoten, sistem nadaljuje delovanje.

2.6.1 Sprotna zaustavitev ob zaznavi človeka

Ko je varnostni režim vključen, sem moral zagotoviti, da motorja takoj dobita nevtralni pulz. Ker program deluje v več nitih hkrati, sem želel, da se kosilnica ustavi ne glede na trenutno obremenitev drugih delov sistema. Zato sem v glavni krmilni zanki ob zaznavi cilja ali ob aktivnem varnostnem zadržanju obema izhodoma takoj nastavil nevtralno vrednost. Tako sem dosegel hiter in zanesljiv odziv.

```

if autopilot_mode and (target_detected or safety_latched):
    pi.set_servo_pulsewidth(CH1_OUT, NEUTRAL_PULSE)
    pi.set_servo_pulsewidth(CH2_OUT, NEUTRAL_PULSE)
else:
    pi.set_servo_pulsewidth(CH1_OUT, ch1)
    pi.set_servo_pulsewidth(CH2_OUT, ch2)

```

Izsek kode 7: Samodejna zaustavitev ob zaznani tarči.

2.6.2 Clear-delay logika

V tem delu je opisan sistem odločanja, ali varnostni mehanizem ostane aktiven ali ne. Program v spremenljivko zapiše čas, ko je bila ovira nazadnje zaznana, nato pa preverja, ali je od tega trenutka pretekla vsaj ena sekunda. Če ovire v tem času ni več, se varnostni mehanizem sprosti in program nadaljuje normalno delovanje. S tem sem preprečil, da bi se sistem ob kratkotrajni prekinitvi zaznave prehitro ponovno zagnal.

```
if target_detected:
    safety_latched = True
    target_clear_since = None
else:
    if safety_latched:
        if target_clear_since is None:
            target_clear_since = now
        elif (now - target_clear_since) >= CLEAR_DELAY_S:
            safety_latched = False
            target_clear_since = None
```

Izsek kode 8: 1-sekundni zamik pri ponovni sprostitvi varnostnega stopa.

2.6.3 Zaznava objektov s MobileNetSSD

Uporabil sem MobileNetSSD, ker sem ugotovil, da je za Raspberry Pi pomembnejša hitrost kot maksimalna natančnost. YOLO sem preizkusil, vendar je bil za moj sistem prepočasen, saj je prepoznavna trajala približno dve sekundi, kar bi v praksi pomenilo prepozen odziv. Sliko pri prepoznavi zmanjšam na 160×160 pikslov, saj je tako obdelava hitrejša. Glede na teste se natančnost zaradi tega ni toliko zmanjšala, da bi sistem postal neuporaben. Pri takšnih sistemih je treba poiskati primerno razmerje med natančnostjo in hitrostjo obdelave slike.

```
small = cv2.resize(current_frame, (160, 160))
blob = cv2.dnn.blobFromImage(small, 0.007843, (160, 160), 127.5)

net.setInput(blob)
detections = net.forward()

detected_now = any(
    float(detections[0, 0, i, 2]) >= CONF_THRESH and
    CLASSES[int(detections[0, 0, i, 1])] in TARGETS
    for i in range(detections.shape[2])
)
```

Izsek kode 9: Zaznavanje tarč.

2.6.4 Zmanjšanje zakasnitve video toka

Video tok sem namensko stisnil in zmanjšal kakovost slike, ker sem ugotovil, da je za varnostni prikaz pomembnejša odzivnost kot popolna ostrina vsakega okvirja. Sprva se je slika posodabljala približno vsakih 1,5 s, zato sem zmanjšal ločljivost in omogočil kompresijo, da se je video na

strežniku, do katerega sem dostopal prek telefona, posodabljal s približno 0,5 s zamika. To je bilo dovolj hitro za varno vožnjo na daljavo.

```
ok, jpeg = cv2.imencode('.jpg', current_frame,
                        [cv2.IMWRITE_JPEG_QUALITY, 45])
if ok:
    self.wfile.write(b'--frame\r\n')
    self.wfile.write(b'Content-Type: image/jpeg\r\n\r\n')
    self.wfile.write(jpeg.tobytes())
    self.wfile.write(b'\r\n')
```

Izsek kode 10: MJPEG-strežnik z zmerno kakovostjo JPEG.

2.7 GNSS, DGPS in NTRIP v praksi

2.7.1 Delovanje GPS in omejitve navadnih modulov

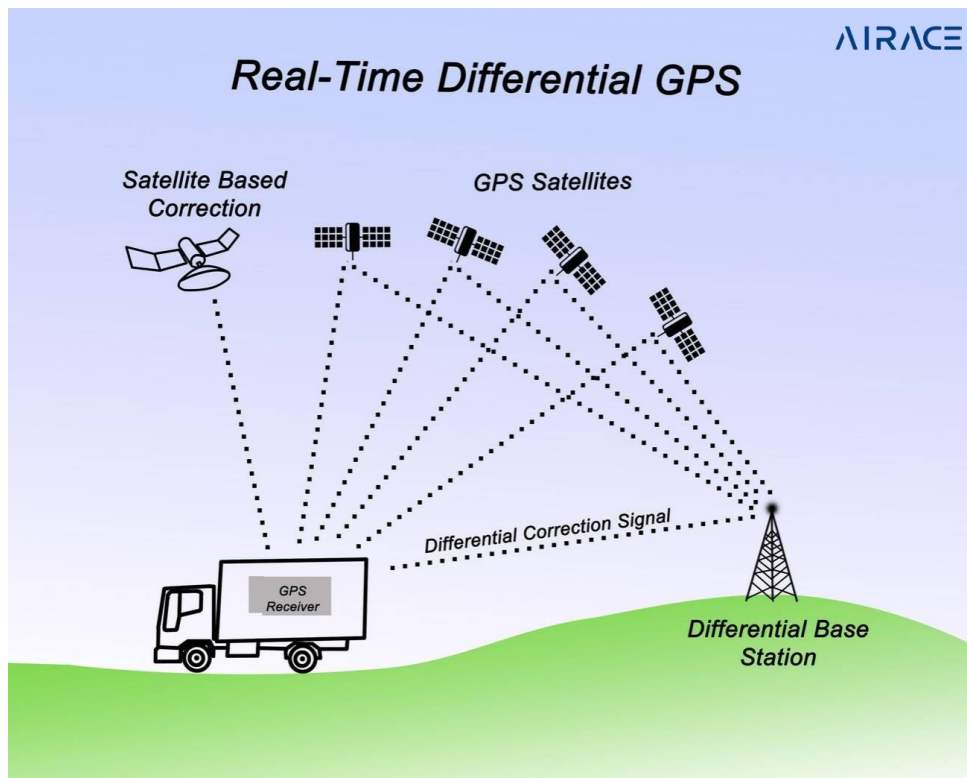
GPS deluje na podlagi izmere časa, ki ga signal potrebuje, da potuje od satelita do sprejemnika. Ker signal potuje s svetlobno hitrostjo, sprejemnik iz razlik v časih sprejema od vsaj štirih satelitov izračuna svoj prostorski položaj. Natančnost je odvisna od geometrije satelitov, atmosferskih pojavov, odbojev signalov in kakovosti sprejemnika. Evropska vesoljska agencija v svojih navodilih za satelitsko navigacijo poudarja, da je za višjo natančnost potrebna dodatna korekcijska plast [1].

Navadni ceneni GPS-moduli dosegajo natančnost okrog ± 5 m, ki pa se lahko nenadoma poslabša na 30 m ali več, kadar imamo omejen pogled na nebo ali ko signal odbijajo okoliški predmeti. Za kosilnico, ki jo operater vodi na daljavo in je morda ne vidi neposredno, je takšna nenatančnost položaja nezanesljiva in potencialno nevarna. Zato sem moral poiskati natančnejšo rešitev.

2.7.2 Diferencialni GPS in NTRIP

Diferencialni GPS (DGPS) deluje tako, da referenčna postaja na znani lokaciji neprestano meri napako GPS-a in te popravke v obliki RTCM-sporočil pošilja sprejemniku v terenu. Sprejemnik nato odšteje izmerjeno napako in dobi bistveno bolj stabilen položaj. Uradna dokumentacija u-blox opisuje GNSS kot skupino sistemov za določanje položaja z več satelitskimi konstelacijami [7].

V mojem projektu sem za popravke uporabil javno dostopno storitev RTK2go in referenčno postajo FRELIH v Črnem Vrhu nad Idrijo [6]. Testiranja sem izvajal v Cerknem, ki je od postaje oddaljeno okrog 33 km, kar je bilo za moje potrebe še uporabno.



Slika 7: Prikaz principa delovanja diferencialnega GPS in popravkov NTRIP.

2.7.3 Povezava na NTRIP in pošiljanje GGA

Sprejemnik prek serijskega vmesnika pošilja NMEA-stavke, med katerimi je ključen GGA (*Global Positioning System Fix Data*). Tega pošiljamo nazaj na NTRIP-caster, ki v zameno vrne RTCM-korekcijske podatke.

Za takšno delovanje je potrebna stalna povezava z internetom. Zato sem sistem med testiranjem povezal na dostopno točko mobilnega telefona, ki je bil povezan v mobilno omrežje. Druga možnost bi bila uporaba ločenega mobilnega Wi-Fi-modema, ki bi omogočal enako funkcionalnost tudi brez telefona.

```
sock = socket.create_connection((NTRIP_HOST, NTRIP_PORT),
                               timeout=10)

auth = base64.b64encode(
    f"{USERNAME}:{PASSWORD}".encode()).decode()
req = (
    f"GET /{MOUNTPOINT} HTTP/1.0\r\n"
    f"User-Agent: Python-NTRIP\r\n"
    f"Authorization: Basic {auth}\r\n\r\n"
)
sock.sendall(req.encode("ascii", errors="ignore"))
```

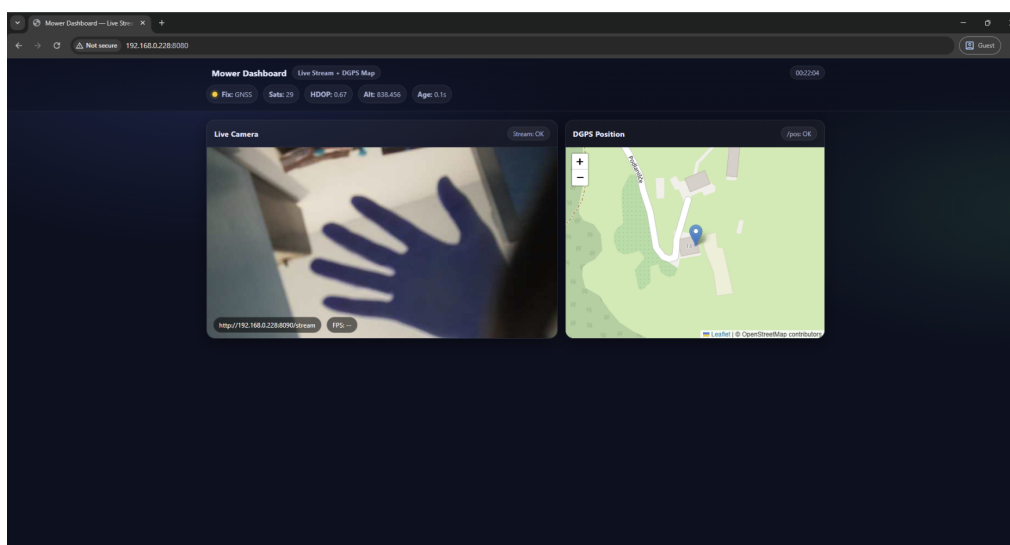
Izsek kode 11: Vzpostavitev povezave na NTRIP-caster in pošiljanje GGA-stavka.

V praksi sem ugotovil, da se natančnost z DGPS-popravki izboljša dovolj, da je položaj dejansko uporaben za orientacijo pri daljinskem delu.

2.8 Spletni vmesnik za operaterja

Spletni vmesnik sem izdelal zato, da sem lahko sistem spremljal brez neposrednega priklopa z kablom na Raspberry Pi in brez branja surovih podatkov prek terminala. Povezava prek omrežja Wi-Fi je to bistveno poenostavila. Kosilnica v živo pretaka video s kamere in pošilja svoj položaj, ki se nato prikaže na odprtokodnem zemljevidu Leaflet. Vmesnik sem razvil v HTML-ju in JavaScriptu brez dodatnih strežniških ogrodij, ker sem želel, da sistem ostane čim bolj preprost za vzdrževanje.

Ugotovil sem, da sta video slika in prikaz položaja skupaj veliko bolj uporabna kot zgolj surovi podatki. Operater lahko takoj vidi, ali je sistem na pravem mestu in ali je kosilnica pravilno usmerjena.



Slika 8: Spletni vmesnik z živim videoposnetkom kamere in GPS-lokacijo kosilnice na interaktivnem zemljevidu.

2.9 Testiranje sistema in analiza rezultatov

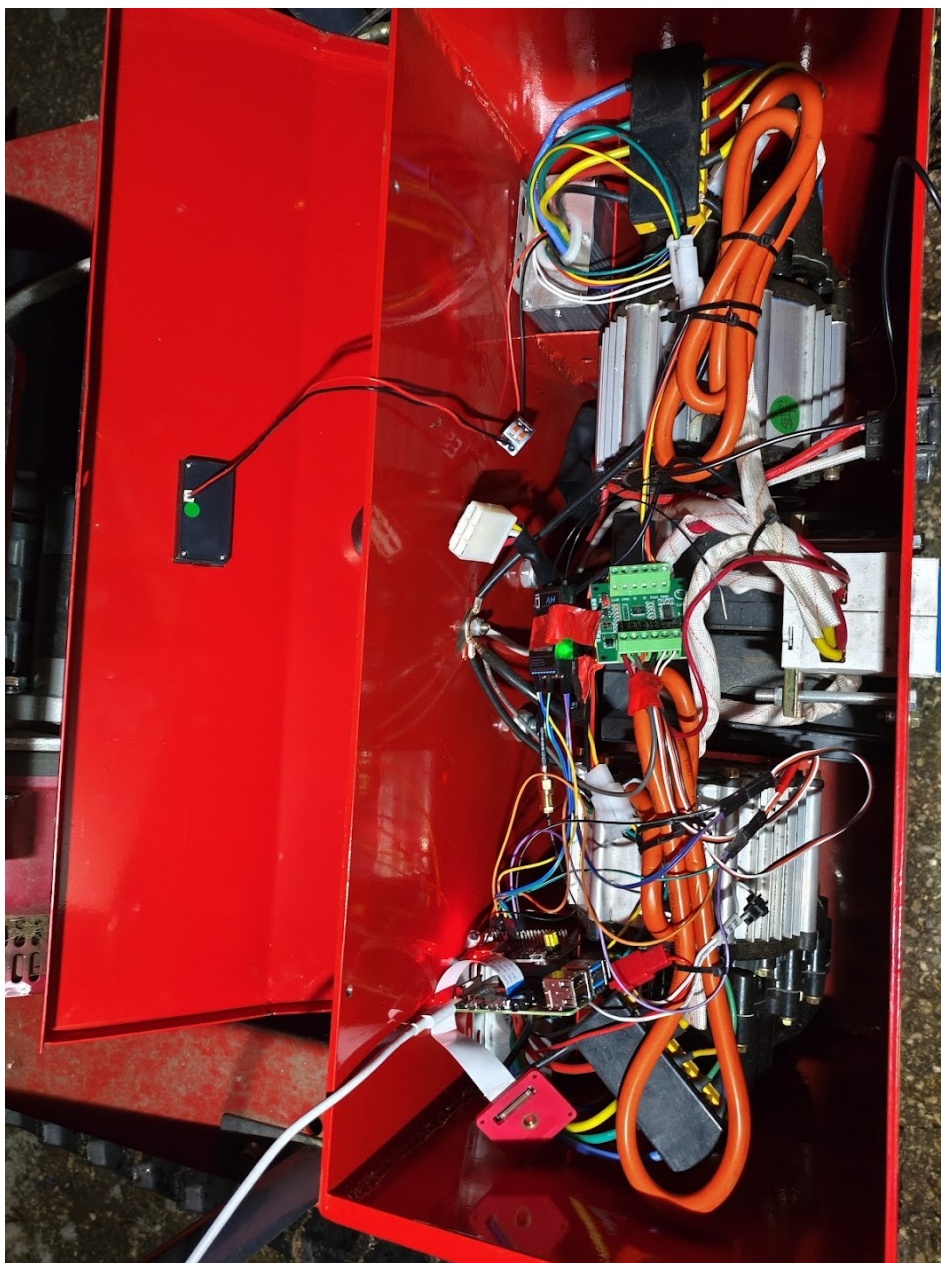
Preden sem združil vse v končno skripto, sem moral posamezne dele preizkusiti ločeno. Zato sem uporabil skripto za kamero, prikaz PWM-signalov ter ločen preizkus NTRIP-povezave. To mi je pomagalo, da sem odkril težave, preden so postale del večje skripte.

Ena od napak, ki sem jo našel, je bila, da se je kamera pri preveliki ločljivosti prepočasi odzivala. Zato sem zmanjšal velikost vhodne slike in inferenco izvajal samo na vsakem drugem okviru. Druga težava je bila, da sem moral zelo natančno določiti prage za CH5, ker se je sicer način delovanja nepredvidljivo spreminjal med varnostnim in običajnim režimom. Pri GPS-delu pa sem ugotovil, da brez RTK-popravljen položaj preveč niha, zato sem uporabil NTRIP-povezavo za natančnejšo lokacijo.

Med terenskim testiranjem sem preveril odzivni čas od zaznave do ustavitve ter stabilnost GPS-položaja. Sistem je zaustavil kosilnico v manj kot 100 ms od zaznave osebe pri razdalji do 4 m in zadostnih svetlobnih razmerah. GPS-natančnost je bila z DGPS-popravki stabilna pod 1 m, v primerjavi s ± 5 m ali več brez popravkov.



Slika 9: Testiranje sistema na terenu: mulčer od daleč s prenosnim računalnikom med preizkusi.



Slika 10: Notranjost mulčerja z dodanim Raspberry Pi računalnikom in nameščeno elektroniko.



Slika 11: Daljinec, nameščen na mulčerju med terenskim testiranjem.

3 Zaključek

V seminarski nalogi sem predstavil nadgradnjo daljinsko vodene kosilnice z računalniškim vidom, spletnim vmesnikom in natančnejšim določanjem položaja. Projekt sem zasnoval od izbire strojne opreme in programske arhitekture do implementacije glavnih funkcionalnosti, kot so zaznavanje ljudi in živali, samodejna zaustavitve, prenos video slike in prikaz lokacije na zemljevidu.

Pri delu sem ugotovil, da so si posamezni deli sistema na prvi pogled videti razmeroma preprosti, v praksi pa se pri vsakem pojavijo nove težave. Posebej pomembno je bilo uskladiti hitrost zaznavanja, zakasnitev video prenosa in zanesljivost varnostne logike. Pokazalo se je, da pri takšnem sistemu ni najpomembnejša samo teoretična natančnost, temveč predvsem dovolj hiter in zanesljiv odziv v resničnem okolju.

Skozi razvoj projekta sem pridobil veliko praktičnih izkušenj s področja računalniškega vida, večnitnega programiranja, GNSS-lociranja, spletnih vmesnikov in povezovanja elektronskih modulov v enoten sistem. Pomembno je bilo tudi to, da sem posamezne podsisteme najprej testiral ločeno in jih šele nato združil v končni program `final.py`.

Rezultati kažejo, da sem zastavljeni cilj v veliki meri dosegel. Sistem zna zaznati nevarno bližino človeka ali živali in kosilnico pravočasno ustaviti, poleg tega pa operaterju omogoča spremljanje slike in položaja na daljavo. Kljub temu ostajajo možnosti za nadaljnjo nadgradnjo. Za večjo natančnost zaznavanja bi lahko uporabil novejši modul Raspberry Pi Camera AI, ki ima namensko strojno podporo za obdelavo slike že na samem kamerinem modulu, ali zmogljivejši računalnik. Sistem bi bilo mogoče izboljšati tudi z dodatno osvetlitvijo za delo v slabših svetlobnih razmerah ter z nadaljnjo optimizacijo zakasnitev.

Projekt mi predstavlja trdno osnovo za nadaljnji razvoj in nadgrajevanje, hkrati pa sem z njim pokazal, da je mogoče tudi z razmeroma dostopno opremo izdelati uporaben in varnostno smiseln sistem za podporo pri daljinskem upravljanju delovnega stroja.

4 Viri in literatura

- [1] European Space Agency. *Satellite navigation*. 2026. URL: https://www.esa.int/Applications/Satellite_navigation (pridobljeno 5. 4. 2026).
- [2] Microzone. *MC8RE-V2 user manual*. 2021. URL: <http://microzone.cn/notes/MC8RE-V2/MC8RE-V2%E8%AF%B4%E6%98%8E%E4%B9%A6%EF%BC%88%E8%8B%B1%E6%96%87%EF%BC%89.pdf> (pridobljeno 5. 4. 2026).
- [3] Luka Polc. *Elektronski sistem daljinsko vodene kosilnice za zahtevne terene*. Maribor: Univerza v Mariboru, Fakulteta za elektrotehniko, računalništvo in informatiko, 2013.
- [4] Raspberry Pi Ltd. *Camera*. 2026. URL: <https://www.raspberrypi.com/documentation/accessories/camera.html> (pridobljeno 5. 4. 2026).
- [5] Raspberry Pi Ltd. *Camera software*. 2026. URL: https://www.raspberrypi.com/documentation/computers/camera_software.html (pridobljeno 5. 4. 2026).
- [6] RTK2go. *RTK2go NTRIP caster*. 2026. URL: <https://rtk2go.com/> (pridobljeno 5. 4. 2026).
- [7] u-blox. *GNSS*. 2026. URL: <https://www.u-blox.com/en/technologies/gnss> (pridobljeno 5. 4. 2026).

Izjava o avtorstvu

Izjavljam, da je seminarska naloga v celoti moje avtorsko delo, ki sem ga izdelal samostojno s pomočjo navedene literature in pod vodstvom mentorja.

6. april 2026

Lin Čadež